Bu-Ali Sina University

Unmatched (phase 1)

Mohadese Nejatbakhsh *40412358054*

Dina Sharifypanah *40412358025*

Dr. Yousef-Sanati

Table of Contents

introduction

Unmatched in our project is a two-player tactical board game in which legendary heroes from literature, history, and popular culture battle against one another using unique abilities and custom card decks. Each hero has a distinct playstyle, making every match different and requiring players to adapt their strategies based on the chosen characters.

In this project, the game features **Sherlock Holmes** and **Count Dracula** as the playable heroes. Each hero is accompanied by their own sidekicks and possesses special abilities that influence movement, combat, and card effects throughout the game.

The objective of the game is to defeat the opponent's hero through careful planning, efficient use of action points, tactical movement across the map, and intelligent management of attack, defense, versatile, and scheme cards. Players must make strategic decisions every turn, considering their fighter positions, available cards, and hero abilities to gain an advantage.

This project implements a terminal-based version of Unmatched using **C++** and object-oriented programming principles. It models the game's core mechanics, including the

game board, fighters, cards, combat system, player turns, and unique hero abilities. To provide a more interactive user experience, the project also utilizes the **FTXUI** library to create a modern text-based user interface with navigable menus.

Game Map

The game **map** is managed by the Board class. The board consists of 32 spaces, each represented by a Space object. The Space class stores all the information related to a single home on the map, including its unique ID, neighboring spaces, zone(s), whether it contains a secret passage, and the fighter currently occupying it.

The Board class provides several utility functions such as checking neighboring spaces, verifying whether two spaces belong to the same zone, finding empty spaces, retrieving spaces in a specific zone, moving fighters, and swapping the positions of two fighters.

```
vector<Space *> findSpacesByZone(ZoneType zone) const;
vector<Space *> findEmptySpaces() const;

void addSpace(Space *space);
void setupMap();
void connectSpaces(int firstId, int secondId);
```

To calculate all legal movement destinations, the getAvailableMoves() function is implemented using the Breadth-First Search (BFS) algorithm. a queue is used to explore spaces level by level based on the fighter's movement value, while a set keeps track of visited spaces to prevent revisiting the same location.

```cpp
vector<Space *> Board::getAvailableMoves(Fighter *fighter, int maxStep) const
{
    vector<Space *> result;
    if (fighter == nullptr)
    {
        return result;
    }
    Space *start = fighter->getPosition();
    if (start == nullptr)
    {
        return result;
    }
    queue<pair<Space *, int>> q;
    set<Space *> visited;

    q.push({start, 0});
    visited.insert(start);

    while (!q.empty())
    {
        Space *current = q.front().first;
        int distance = q.front().second;
        q.pop();

        if (distance == maxStep)
        {
            continue;
        }
        vector<Space *> nextSpaces = current->getNeighbors();
```

```cpp
        if (current->hasSecretPassage())
        {
            for (auto space : spaces)
            {
                if (space != current && space->hasSecretPassage())
                {
                    nextSpaces.push_back(space);
                }
            }
        }
        for (auto next : nextSpaces)
        {
            if (visited.count(next))
            {
                continue;
            }
            visited.insert(next);

            Fighter *user = next->getFighter();
            if (user == nullptr)
            {
                result.push_back(next);
                q.push({next, distance + 1});
                continue;
            }
            if (user->getOwner() == fighter->getOwner())
            {
                q.push({next, distance + 1});
                continue;
            }
```

```cpp
            if (user->getOwner() == fighter->getOwner())
            {
                q.push({next, distance + 1});
                continue;
            }
        }
    }
    return result;
}
```

Card files

The card management subsystem is one of the most important parts of the project because almost every gameplay mechanic depends on it. It is responsible for creating cards, storing them inside the deck, managing the player's hand, handling discarded cards, and executing special card effects during combat. To achieve this, the project is divided into four main classes: **Card**, **Deck**, **Hand**, and **Discard Pile**. Each class has a specific responsibility that follows the principle of separation of concerns.

Card class

The **Card** class represents a single game card. Every card contains all of the information required during gameplay, including:

- Card name
- Card type (Attack, Defense, Versatile, or Scheme)
- Card owner (Hero, Sidekick, or Any)
- Combat value
- Boost value
- Timing of the effect
- Number of copies in the deck
- Pointer to a special Effect object

Besides storing data, this class is responsible for validating whether a card can be played by a specific fighter and displaying the complete card information to the user.

Important Functions

Play ()

This function is called whenever a card is played during combat. It first checks whether the selected fighter is allowed to use the card and then displays the card information before combat resolution begins.

```
54
55      virtual void play(Fighter &fighter, Fighter &target, Game &game) const;
56      bool canBePlayedBy(const Fighter &fighter, CardType requiredType) const;
57      bool hasEffect() const;
```

isPlayableBy ()

Determines whether the selected fighter is allowed to use the card according to the owner type (Hero, Sidekick, or Any).

canBePlayedBy ()

Checks both the required combat card type and the ownership restriction before allowing the player to select a card.

hasEffect ()

Returns whether the card contains a special effect object.

Factory Methods

Instead of manually constructing cards throughout the project, every card is created using dedicated factory methods such as:

- createFeedingFrenzy ()
- createMistForm ()
- createDash ()
- createCounterPunch ()
- createEducationNeverEnds ()

Each factory method builds a fully initialized card with the correct statistics, timing, owner type, combat values, and associated Effect object.

This approach centralizes card creation, reduces duplicated code, and makes future modifications significantly easier.

Deck class

The **Deck** class represents a player's draw pile. It stores all available cards before the game begins and provides the operations required for drawing and shuffling cards.

Important Functions

drawCard ()

Removes and returns the top card of the deck.

If the deck becomes empty, a null pointer is returned.

```
16
17          void shuffle();
18          Card *drawCard();
19          void addCopies(const Card &card, int count);
20
```

Shuffle ()

Randomly shuffles all cards using the C++ <random> library before the game starts.

addCopies ()

Creates multiple copies of the same card using the copy constructor. This greatly simplifies deck construction.

Hand class

The **Hand** class stores every card currently available to the player.

Unlike the deck, cards inside the hand can be selected, inspected, and played during combat.

Important Functions

addCard ()

Adds a newly drawn card into the player's hand.

removeCard ()

Removes the selected card from the hand and returns its pointer.

This function is used whenever a player plays a combat card.

```
16
17      void addCard(Card *card);
18      Card *removeCard(int index);
19      bool isEmpty() const;
20
```

isEmpty ()

Checks whether the player currently has any cards available.

DiscardPile class

The **DiscardPile** class stores every card that has already been played during the game.

Instead of deleting cards immediately after combat, they are transferred into the discard pile where they remain available for future mechanics if needed.

Fighter files

The fighter system is responsible for representing all playable characters in the game. It is designed using inheritance and polymorphism, where the Fighter class serves as the base class for all heroes and sidekicks. Common attributes and behaviors such as health, movement, position, damage handling, and healing are implemented in the base class, while each derived class provides its own unique characteristics and abilities.

Fighter class

Fighter is the abstract base class for every character in the game. It stores common information such as the fighter's name, health, movement value, attack type, owner, and current position on the board.

Important Functions

```
void takeDamage(int damage);
void heal(int amount);
bool isAlive() const;
void moveTo(Space *newSpace);
bool isAdjacent(const Fighter *other, const Board &board) const;
bool isInSameZone(const Fighter *other, const Board &board) const;
```

takeDamage() – Reduces the fighter's health.

heal() – Restores health without exceeding the maximum value.

moveTo() – Moves the fighter to another valid space.

isAdjacent() – Checks whether another fighter is in an adjacent space.

isInSameZone() – Checks whether two fighters are in the same zone.

addTempAttackBoost() / resetTempAttackBoost() – Manages temporary attack bonuses used by card effects.

Hero class

Hero inherits from Fighter and represents the main character controlled by each player. Every hero has a unique special ability and an ability description.

In our implementation, the Hero class has two derived classes:

- Dracula
- Sherlock Holmes

Important Functions

```
const string &getAbilityDescription() const;
virtual void useAbility(Game& game, Player &player);
```

useAbility() – Activates the hero's special ability.

getAbilityDescription() – Returns the hero's ability description

Sidekick class

The Sidekick class also inherits from Fighter and represents the supporting fighters that assist the heroes during the game.

In our implementation, the Sidekick class has two derived classes:

- Dr. Watson (Sherlock Holmes' sidekick)
- Sisters (Dracula's three sidekicks)

These classes mainly define the specific properties of each sidekick, Sisters and Dr.Watson , such as health, movement, attack type, and any special characteristics.

Game module

The **Game module** is the central controller of the entire project. It coordinates all gameplay mechanics and manages the interaction between different modules such as the Board, Players, Fighters, Cards, User Interface, and Effects.

Instead of allowing each component to directly communicate with every other class, the Game module acts as the game engine that controls the overall execution order. This design improves modularity, reduces coupling between classes, and makes the project easier to maintain and extend.

Game class

The **Game** class represents the complete game session. It initializes every subsystem, creates the players and fighters, manages the board, controls the game loop, and determines when the match has finished.

It serves as the highest-level controller of the application.

Important Functions

Run ()

Starts the entire gameplay loop and repeatedly processes player turns until one player's hero is defeated.

Initialize ()

Creates players, heroes, sidekicks, decks, distributes cards, initializes the board, and prepares every component required before gameplay begins.

Endgame ()

Continuously verifies whether one of the heroes has been defeated and determines the winner of the match.

ProcessPlayerAction ()

Receives the player's selected action (Attack, Maneuver, or Scheme) and executes the corresponding game logic.

```
23
24          void initialize();
25          void run();
26          void processPlayerAction();
27          void endGame();
```

CombatSystem Class

The **CombatSystem** class is responsible for implementing the complete combat mechanism of the game. It validates attacks, determines whether two fighters are within attack range, calculates attack and defense values, resolves combat outcomes, applies card effects according to their timing, inflicts damage, and finally sends the played cards to the discard piles. By isolating all combat-related logic inside a dedicated class, the project achieves better modularity and makes the battle mechanics easier to maintain and extend.

Important Functions

resolveCombat ()

Executes the complete combat sequence, including card validation, effect activation, damage calculation, health reduction, and combat result display.

isAttackValid ()

Ensures that an attack satisfies all required conditions before combat begins.

applyEffects ()

Activates card effects according to their timing (**Immediately**, **During Combat**, or **After Combat**) while respecting cancellation mechanics such as **Feint**.

```
29
30      void resolveCombat(Game &game, Fighter &attacker, Fighter &defender,
31                         Card &attackCard, Card *defenceCard);
32      bool isAttackValid() const;
33      void applyEffects(Timing timing, Fighter &user, Fighter &target, Card &card, bool didUserWin);
34
```

TurnManager Class

The **TurnManager** class manages the flow of turns throughout the match. It keeps track of the active player, the waiting player, the current turn number, and the remaining actions available during each turn. Since every player in *Unmatched* is allowed exactly two actions per turn, this class guarantees that players cannot exceed the permitted number of actions and automatically switches control to the opponent when a turn ends.

Important Functions

startGame ()

Initializes the first player, the waiting player, and begins the first turn.

startTurn ()

Resets the available actions to two whenever a new turn begins.

useAction ()

Decreases the remaining action count after each player action.

endTurn ()

Exchanges the active and waiting players, increments the turn counter, and starts the next turn.

hasActions ()

Returns whether the current player still has available actions.

checkHandLimit ()

Verifies that the player's hand size does not exceed the maximum limit defined by the game rules.

```
18
19      void startGame(Player *younger, Player *older);
20      void startTurn();
21      void endTurn();
22      void useAction();
23      bool hasActions() const;
24      bool checkHandLimit() const;
```

Player class

The **Player** class represents one participant in the game and stores all information related to that player. It manages the player's hero, sidekicks, deck, hand, discard pile, and age (used to determine the starting player). By grouping these elements into a single class, all player-related data and operations remain organized and easy to manage throughout the match.

Important Functions

drawCardToHand()

Draws one card from the player's deck and adds it to their hand.

hasPlayableCard()

Checks whether the player has at least one card that can legally be played by a specific fighter.

```
41
42      bool drawCardToHand();
43      bool hasPlayableCard(const Fighter &fighter, CardType requiredType) const;
44      bool isYounger(const Player &other) const;
45
```

isYounger()

Compares the ages of two players to determine who starts the game according to the official rules.

User Interface(UI)

The user interface of the project consists of two main parts: the game interface and the main menu.

TerminalUI Class

The game interface is implemented in the TerminalUI class. It is responsible for displaying all game information while keeping the game logic separate from the presentation layer.

The game board is built from a predefined ASCII map (MAP_TEMPLATE), where each space has a fixed position. During gameplay, the board is updated dynamically by placing fighters on their current locations using unique symbols. The interface also displays the current player's information, hero statistics, sidekick information, remaining actions, hero abilities, and the locations of all friendly fighters.

The TerminalUI class also provides helper functions for rendering the complete screen, displaying the board, and receiving player input such as selecting cards, fighters, and spaces.

Important Functions

```
void drawStaticMap(vector<string> &canvas) const;
void drawHomes(vector<string> &canvas) const;
void drawFighters(vector<string> &canvas,
|   |   |   |   | const vector<Player *> &players) const;
```

drawStaticMap() – Draws the static ASCII map template that serves as the base layout of the game board.

drawHomes() – Draws the IDs of all spaces on the map.drawFighters() – Places fighters on the board according to their current locations.

```
vector<string> buildBoard(const Board &board,
|   |   |   |   |   |   | const vector<Player *> &players) const;
void buildInfoPanel(vector<string> &panel, const Game &game) const;
void buildHeroLocationPanel(vector<string> &panel, const Game &game) const;
void buildSidekickLocationPanel(vector<string> &panel, const Game &game) const;
void renderBoardOnly(const Game& game) const;
vector <string> wrapTextForAbility(const string &text, int width) const;

void renderScreen(const Game &game) const;
string boxLine(const string &text) const;
```

renderScreen() – Displays the complete game interface, including the board and information panels.

buildBoard() – Builds the board by combining the map template with the current fighter positions.

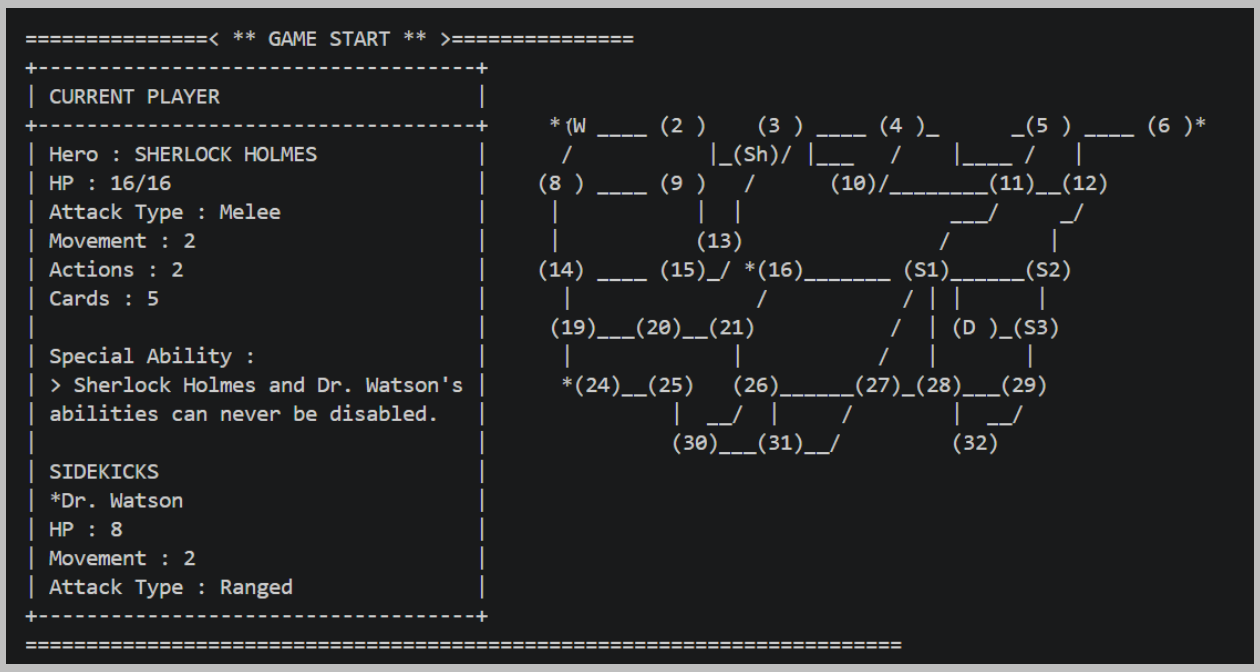
buildInfoPanel() – Creates the information panel for the current player

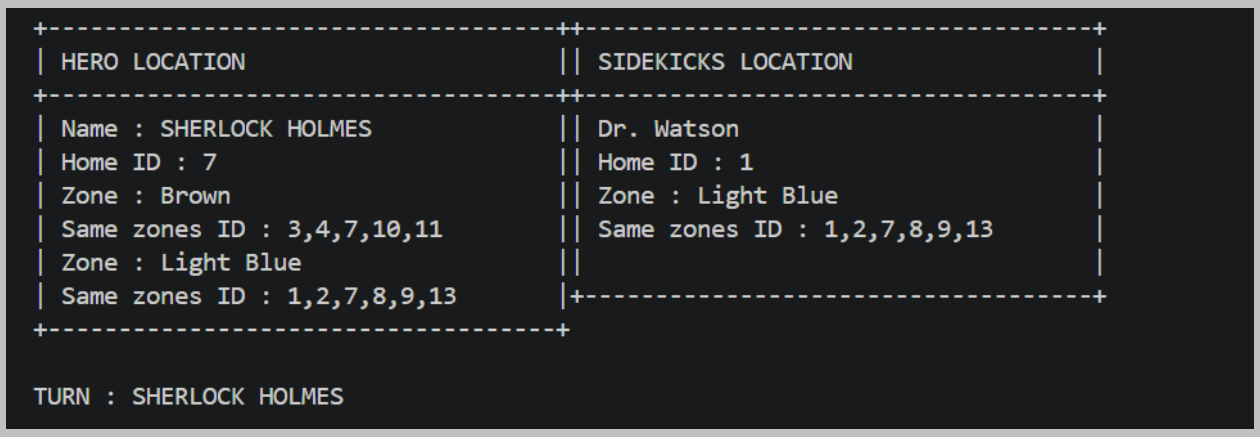
buildHeroLocationPanel() – Displays the hero's current location and zone information.

buildSidekickLocationPanel() – Displays the locations of all sidekicks.

wrapTextForAbility() – Formats long ability descriptions into multiple lines.

Example Output of the Terminal User Interface:





MainMenu Class

The main menu is implemented in the MainMenu class using the FTXUI library. FTXUI provides an interactive terminal interface with modern components such as menus, windows, colors, and keyboard navigation.

The menu contains three options:

Play – Starts a new game.

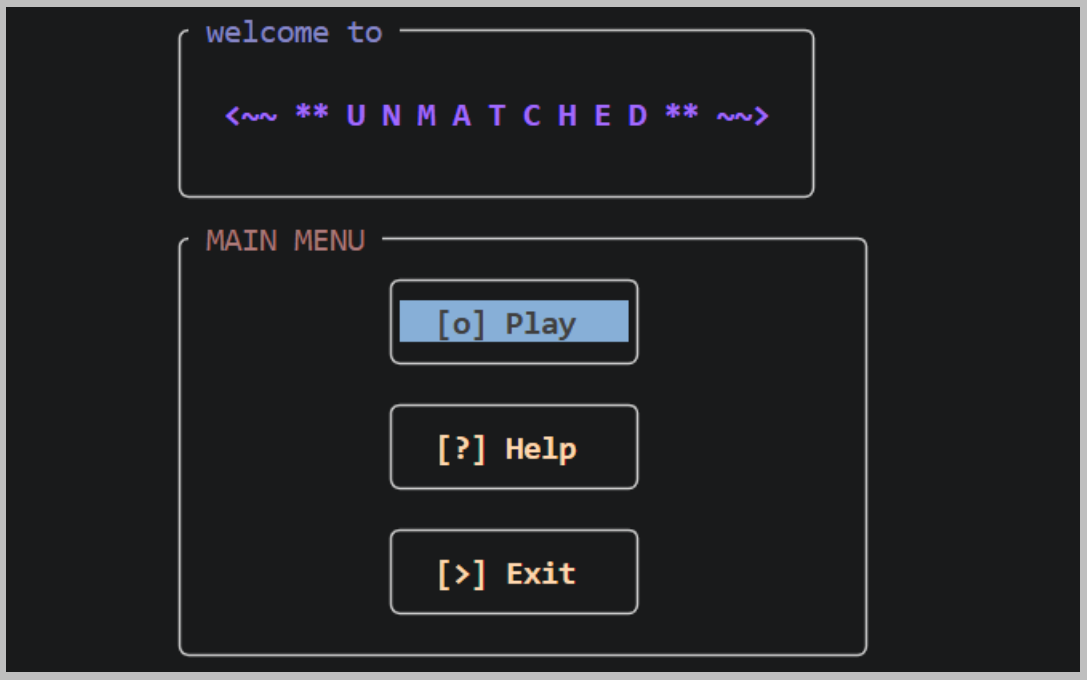
Help – Displays the game instructions and rules.

Exit – Closes the application.

To improve the user experience, the menu is displayed inside bordered windows, uses different colors for the title and buttons, and highlights the currently selected option. Users can navigate the menu using the keyboard and confirm their selection by pressing Enter.

```
4   class MainMenu
5   {
6   public:
7       int show();
8   };
```

Example of the Game's Main Menu:



Types class

The **Types** header defines several enumerations that represent the fundamental categories used throughout the game. These enumerations replace hard-coded values with meaningful names, improving code readability, reducing errors, and making the game logic easier to understand and maintain. Since these types are shared by many classes such as **Card**, **Fighter**, **Board**, and **CombatSystem**, they provide a common language for the entire project.

The enumerations include:

- **AttackType** – Specifies whether a fighter performs **Melee** or **Ranged** attacks.
- **CardType** – Defines the four categories of cards: **Attack**, **Defense**, **Versatile**, and **Scheme**.
- **Timing** – Determines when a card's special effect is activated (**Immediately**, **During Combat**, **After Combat**, or **None**).
- **OwnerType** – Indicates whether a card can be played only by a **Hero**, only by a **Sidekick**, or by **Any** fighter.

- **ZoneType** – Represents the different colored zones on the game board, which are used for movement, positioning, and range calculations.

```
4
5   enum class AttackType
6   {
7       Melee,
8       Ranged
9   };
10
11  enum class CardType
12  {
13      Attack,
14      Defense,
15      Versatile,
16      Scheme
17  };
18
19  enum class Timing
20  {
21      None,
22      Immediately,
23      DuringCombat,
24      AfterCombat
25  };
```

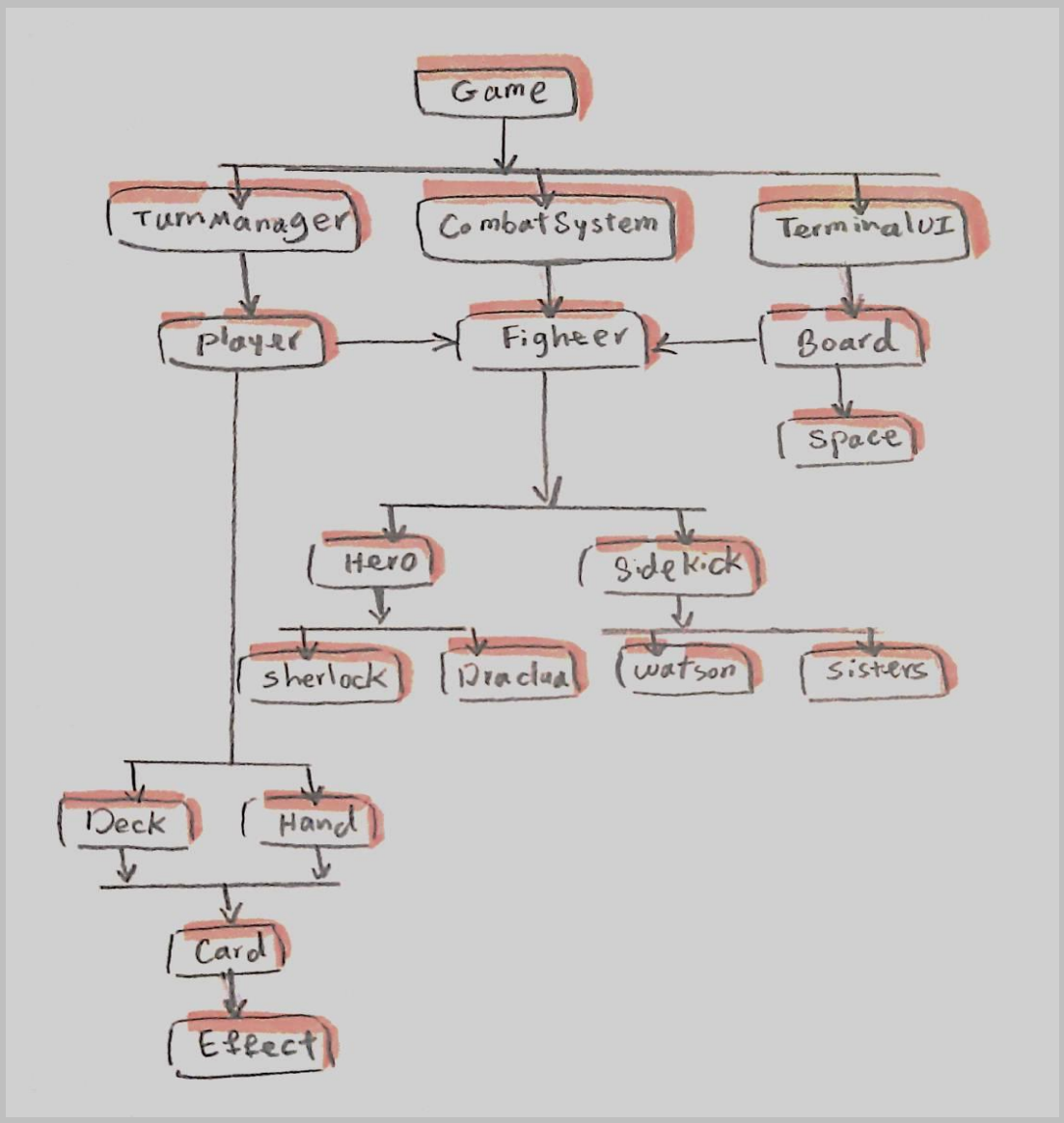
Project Architecture

The project follows an object-oriented architecture where the Game class acts as the central controller of the application. It coordinates the interaction between the game logic, players, combat system, user interface, and the game board.

Each player owns a hero, a collection of sidekicks, and three card containers (Deck, Hand, and DiscardPile). During gameplay, the TurnManager controls the game flow, while the CombatSystem handles all attack, defense, damage calculation, and card effect resolution.

The Board manages spaces and movement rules, and the TerminalUI is responsible for displaying the game state and receiving user input.

Cards are represented by the Card class, while their special abilities are encapsulated in subclasses of the Effect class, making the system modular and easy to extend with new characters and abilities.


Game
TurnManager
CombatSystem
TerminalUI
player
Figheer
Board
Space
Hero
Sidekick
sherlock
Dracula
watson
sisters
Deck
Hand
Card
Effect

Tools

CMAKE: CMake was used as the build system to organize the project, manage source files, and generate build files for different platforms and compilers.

FTXUI: We used the FTXUI library to create an interactive terminal-based user interface. It was used to implement the main menu with selectable buttons, colored text

GIT: Git was used as the version control system to track changes, manage different versions of the project, and simplify the development process.

Challenges

During the development of this project, we faced several challenges:

Debugging: The biggest challenge was debugging the project. Finding and fixing logical errors was often difficult and time-consuming, and we spent nearly half of the project development time debugging different parts of the code.

Project Architecture: Designing the initial architecture of the project was another major challenge. Deciding how to organize the classes and define the relationships between different components required careful planning and several revisions.

Using FTXUI: Learning and integrating the FTXUI library was also challenging. Since we had no prior experience with this library, implementing the interactive main menu, colors, and custom UI elements required additional time and experimentation.

Repository link

The source code, project files, and documentation are available in the following GitHub repository:

https://github.com/momoshinz/Unmatched.git

Resources

The following resources were used during the implementation of this project:

- *C++ How to Program*
- *cppreference.com*
- *chatGPT ai*